Common Problems

Bitly

Scaling Reads

By Evan King · Updated Feb 14, 2026 · easy · Asked at: +18

Watch Video Walkthrough

Watch the author walk through the problem step-by-step

Watch Now

Understanding the Problem

What is Bit.ly?

Bit.ly is a URL shortening service that converts long URLs into shorter, manageable links. It also provides analytics for the shortened URLs.

Designing a URL shortener is a very common beginner system design interview question. Whereas in many of the other breakdowns on Hello Interview we focus on depth, for this one, I'm going to target a more junior audience. If you're new to system design, this is a great question to start with! I'll try my best to slow down and teach concepts that are otherwise taken for granted in other breakdowns.

Functional Requirements

The first thing you'll want to do when starting a system design interview is to get a clear understanding of the requirements of the system. Functional requirements are the features that the system must have to satisfy the needs of the user.

In some interviews, the interviewer will provide you with the core functional requirements upfront. In other cases, you'll need to determine these requirements yourself. If you're familiar with the product, this task should be relatively straightforward. However, if you're not, it's

advisable to ask your interviewer some clarifying questions to gain a better understanding of the system.

The most important thing is that you zero in on the top 3-4 features of the system and don't get distracted by the bells and whistles.

We'll concentrate on the following set of functional requirements:

Core Requirements

1. Users should be able to submit a long URL and receive a shortened version.
    - Optionally, users should be able to specify a custom alias for their shortened URL (ie. "www.short.ly/my-custom-alias")
    - Optionally, users should be able to specify an expiration date for their shortened URL.
2. Users should be able to access the original URL by using the shortened URL.

Below the line (out of scope):

- User authentication and account management.
- Analytics on link clicks (e.g., click counts, geographic data).

> These features are considered "below the line" because they add complexity to the system without being core to the basic functionality of a URL shortener. In a real interview, you might discuss these with your interviewer to determine if they should be included in your design.

Non-Functional Requirements

Next up, you'll want to outline the core non-functional requirements of the system. Non-functional requirements refer to specifications about how a system operates, rather than what tasks it performs. These requirements are critical as they define system attributes like scalability, latency, security, and availability, and are often framed as specific benchmarks—such as a system's ability to handle 100 million daily active users or respond to queries within 200 milliseconds.

Core Requirements

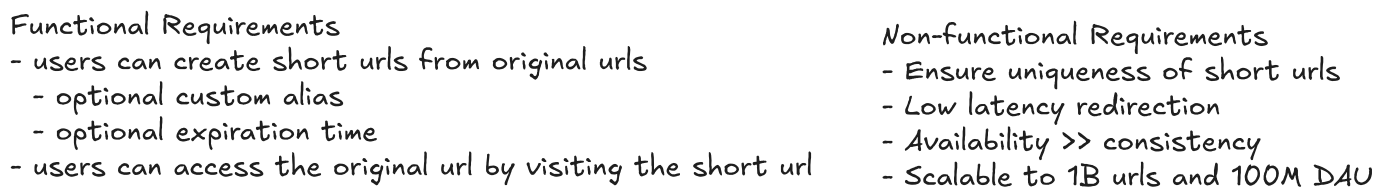
1. The system should ensure uniqueness for the short codes (each short code maps to exactly one long URL)
2. The redirection should occur with minimal delay (< 100ms)
3. The system should be reliable and available 99.99% of the time (availability > consistency)
4. The system should scale to support 1B shortened URLs and 100M DAU

Below the line (out of scope):

- Data consistency in real-time analytics.
- Advanced security features like spam detection and malicious URL filtering.

> An important consideration in this system is the significant imbalance between read and write operations. The read-to-write ratio is heavily skewed towards reads, as users frequently access shortened URLs, while the creation of new short URLs is comparatively rare. For instance, we might see 1000 clicks (reads) for every 1 new short URL created (write). This asymmetry will significantly impact our system design, particularly in areas such as caching strategies, database choice, and overall architecture.

Here is what you might write on the whiteboard:



Bit.ly Non-Functional Requirements

The Set Up

Defining the Core Entities

We recommend that you start with a broad overview of the primary entities. At this stage, it is not necessary to know every specific column or detail. We will focus on the intricacies, such as columns and fields, later when we have a clearer grasp. Initially, establishing these key entities

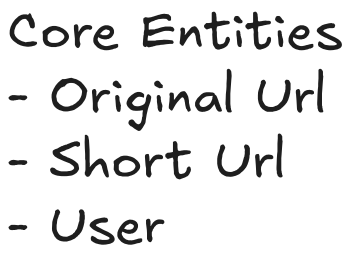
will guide our thought process and lay a solid foundation as we progress towards defining the API.

> Just make sure that you let your interviewer know your plan so you're on the same page. I'll often explain that I'm going to start with just a simple list, but as we get to the high-level design, I'll document the data model more thoroughly.

In a URL shortener, the core entities are very straightforward:

1. **Original URL**: The original long URL that the user wants to shorten.
2. **Short URL**: The shortened URL that the user receives and can share.
3. **User**: Represents the user who created the shortened URL.

In the actual interview, this can be as simple as a short list like this. Just make sure you talk through the entities with your interviewer to ensure you are on the same page.



Bit.ly Entities

The API

The next step in the delivery framework is to define the APIs of the system. This sets up a contract between the client and the server, and it's the first point of reference for the high-level design.

Your goal is to simply go one-by-one through the core requirements and define the APIs that are necessary to satisfy them. Usually, these map 1:1 to the functional requirements, but there are times when multiple endpoints are needed to satisfy an individual functional requirement.

9/10 you'll use a REST API and focus on choosing the right HTTP method or verb to use.

- **POST**: Create a new resource

- **GET**: Read an existing resource
- **PUT**: Update an existing resource
- **DELETE**: Delete an existing resource

To shorten a URL, we'll need a POST endpoint that takes in the long URL and optionally a custom alias and expiration date, and returns the shortened URL. We use post here because we are creating a new entry in our database mapping the long url to the newly created short url.

```
// Shorten a URL
POST /urls
{
  "long_url": "https://www.example.com/some/very/long/url",
  "custom_alias": "optional_custom_alias",
  "expiration_date": "optional_expiration_date"
}
->
{
  "short_url": "http://short.ly/abc123"
}
```

For redirection, we'll need a GET endpoint that takes in the short code and redirects the user to the original long URL. GET is the right verb here because we are reading the existing long url from our database based on the short code.

```
// Redirect to Original URL
GET /{short_code}
-> HTTP 302 Redirect to the original long URL
```

> We'll talk more about which HTTP status codes to use during the high-level design.

High-Level Design

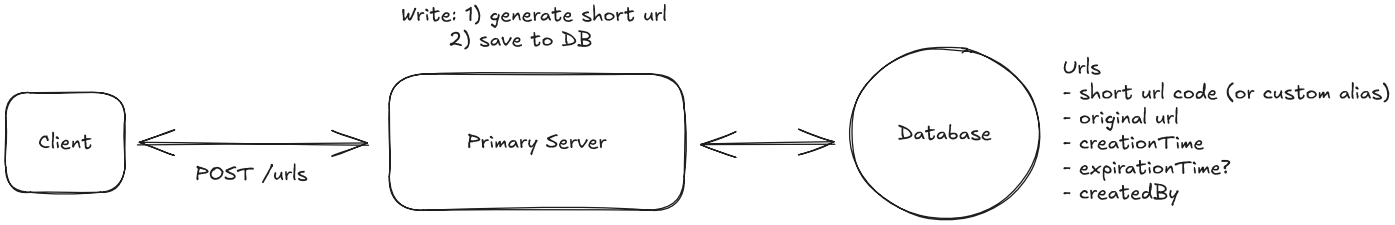
We'll start our design by going one-by-one through our functional requirements and designing a single system to satisfy them. Once we have this in place, we'll layer on depth via our deep

dives.

1) Users should be able to submit a long URL and receive a shortened version

The first thing we need to consider when designing this system is how we're going to generate a short url. Users are going to come to us with long urls and expect us to shrink them down to a manageable size.

We'll outline the core components necessary to make this happen at a high-level.



Create a short url

1. **Client**: Users interact with the system through a web or mobile application.
2. **Primary Server**: The primary server receives requests from the client and handles all business logic like short url creation and validation.
3. **Database**: Stores the mapping of short codes to long urls, as well as user-generated aliases and expiration dates.

When a user submits a long url, the client sends a POST request to `/urls` with the long url, custom alias, and expiration date. Then:

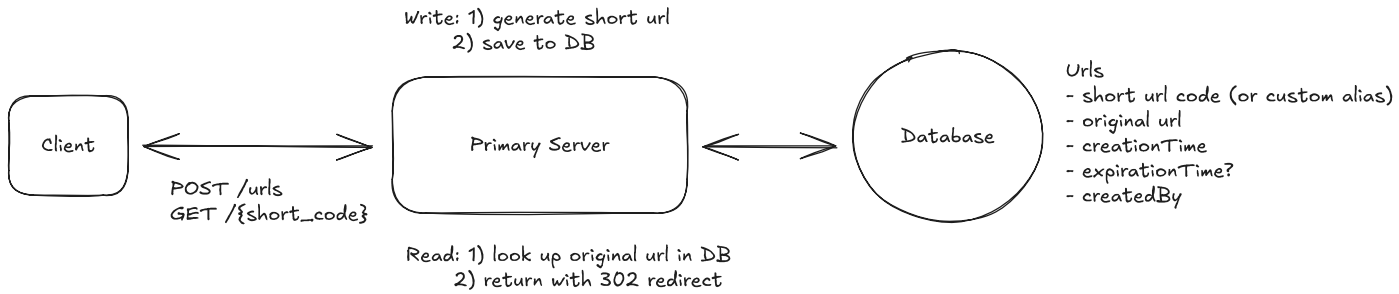
1. The Primary Server receives the request and validates the long URL format using libraries like **is-url** or simple validation. Optionally, we can check if this exact long URL was already shortened and return the existing short code (deduplication). However, most URL shorteners allow multiple short codes for the same long URL since different users may want separate expiration dates, independent analytics, or different custom aliases. Deduplication trades off storage efficiency for these features.
2. If the URL is valid, we generate a short code
    - For now, we'll abstract this away as some magic function that takes in the long URL and returns a short URL. We'll dive deep into how to generate short URLs in the next section.

- If the user has specified a custom alias, we can use that as the short code (after validating that it doesn't already exist). To prevent custom aliases from colliding with future counter-generated codes, consider prefixing generated codes with a character that custom aliases can't use, or store them in separate namespaces.

3. Once we have the short URL, we can proceed to insert it into our database, storing the short code (or custom alias), long URL, and expiration date.
4. Finally, we can return the short URL to the client.

2) Users should be able to access the original URL by using the shortened URL

Now our short URL is live and users can access the original URL by using the shortened URL. Importantly, this shortened URL exists at a domain that we own! For example, if our site is located at `short.ly`, then our short urls look like `short.ly/abc123` and all requests to that short url go to our Primary Server.



Redirect to original url

When a user accesses a shortened URL, the following process occurs:

1. The user's browser sends a GET request to our server with the short code (e.g., `GET /abc123`).
2. Our Primary Server receives this request and looks up the short code (`abc123`) in the database.
3. If the short code is found and hasn't expired (by comparing the current date to the expiration date in the database), the server retrieves the corresponding long URL. For expired URLs, return a 410 Gone status.
4. The server then sends an HTTP redirect response to the user's browser, instructing it to navigate to the original long URL.

For cleanup, we can run a background job periodically to delete expired rows from the database (or just keep them with their expiration date). More importantly, we should set the cache TTL to match or be shorter than URL expiration times so stale entries are automatically evicted.

There are two main types of HTTP redirects that we could use for this purpose:

1. **301 (Permanent Redirect)**: This indicates that the resource has been permanently moved to the target URL. Browsers typically cache this response, meaning subsequent requests for the same short URL might go directly to the long URL, bypassing our server.

The response back to the client looks like this:

```
HTTP/1.1 301 Moved Permanently
Location: https://www.original-long-url.com
```

2. **302 (Found)**: This indicates that the resource is temporarily located at a different URL. Browsers do not cache this response, ensuring that future requests for the short URL will always go through our server first.

The response back to the client looks like this:

```
HTTP/1.1 302 Found
Location: https://www.original-long-url.com
```

In either case, the user's browser (the client) will automatically follow the redirect to the original long URL and users will never even know that a redirect happened.

For a URL shortener, a 302 redirect is often preferred because:

- It gives us more control over the redirection process, allowing us to update or expire links as needed.
- It prevents browsers from caching the redirect, which could cause issues if we need to change or delete the short URL in the future.
- It allows us to track click statistics for each short URL (even though this is out of scope for this design).

Potential Deep Dives

At this point, we have a basic, functioning system that satisfies the functional requirements. However, there are a number of areas we could dive deeper into to reduce the likelihood of collision, support scalability, and improve performance. We can now look back at our non-functional requirements and see which ones still need to be satisfied or improved upon.

1) How can we ensure short urls are unique?

In our high-level design, we abstracted away the details of how we generate a short url but now it's time to get into the nitty-gritty! There are a handful of constraints we need to keep in mind as we design:

1. We need to ensure that the short codes are unique.
2. We want the short codes to be as short as possible (it is a url shortener afterall).
3. We want to ensure codes are efficiently generated.

Let's weigh a few options and consider their pros and cons.

Bad Solution: Long Url Prefix

Great Solution: Hash Function

Great Solution: Unique Counter with Base62 Encoding

2) How can we ensure that redirects are fast?

When dealing with a large database of shortened URLs, finding the right match quickly becomes crucial for a smooth user experience. Without any optimization, our system would need to check every single pair of short and original URLs in the database to find the one we're looking for. This process, known as a "full table scan," can be incredibly slow, especially as the number of URLs grows into the millions or billions.

Pattern: Scaling Reads

URL shorteners showcase the extreme read-to-write ratio that makes **scaling reads** critical. With potentially millions of clicks per shortened URL, aggressive caching strategies become essential.

Learn This Pattern

Good Solution: Add an Index

Great Solution: Implementing an In-Memory Cache (e.g., Redis)

Great Solution: Leveraging Content Delivery Networks (CDNs) and Edge Computing

3) How can we scale to support 1B shortened urls and 100M DAU?

We've done much of the hard work to scale already! We introduced a caching layer which will help with read scalability, now lets talk a bit about scaling writes.

We'll start by looking at the size of our database.

Each row in our database consists of a short code (~8 bytes), long URL (~100 bytes), creationTime (~8 bytes), optional custom alias (~100 bytes), and expiration date (~8 bytes). This totals to ~200 bytes per row. We can round up to 500 bytes to account for any additional metadata like the creator id, analytics id, etc.

If we store 1B mappings, we're looking at `500 bytes * 1B rows = 500GB` of data. The reality is, this is well within the capabilities of modern SSDs. Given the number of urls on the internet is our maximum bound, we can expect it to grow but only modestly. If we were to hit a hardware limit, we could always shard our data across multiple servers but a single Postgres instance, for example, should do for now.

So what database technology should we use?

The truth is: most will work here. We offloaded the heavy read throughput to a cache and write throughput is pretty low. We could estimate that maybe 100k new urls are created per day.

100k new rows per day is ~1 row per second. So any reasonable database technology should do (ie. Postgres, MySQL, DynamoDB, etc). In your interview, you can just pick whichever you have the most experience with! If you don't have any hands on experience, go with Postgres.

But what if the DB goes down?

It's a valid question, and one always worth considering in your interview. We could use a few different strategies to ensure high availability.

1. **Database Replication**: By using a database like Postgres that supports replication, we can create multiple identical copies of our database on different servers. If one server goes down, we can redirect to another. This adds complexity to our system design as we now need to ensure that our Primary Server can interact with any replica without any issues. This can be tricky to get right and adds operational overhead.
2. **Database Backup**: We could also implement a backup system that periodically takes a snapshot of our database and stores it in a separate location. This adds complexity to our system design as we now need to ensure that our Primary Server can interact with the backup without any issues. This can be tricky to get right and adds operational overhead.

Now, let's point our attention to the Primary Server.

Coming back to our initial observation that reads are much more frequent than writes, we can scale our Primary Server by separating the read and write operations. This introduces a microservice architecture where the Read Service handles redirects while the Write service handles the creation of new short urls. This separation allows us to scale each service independently based on their specific demands.

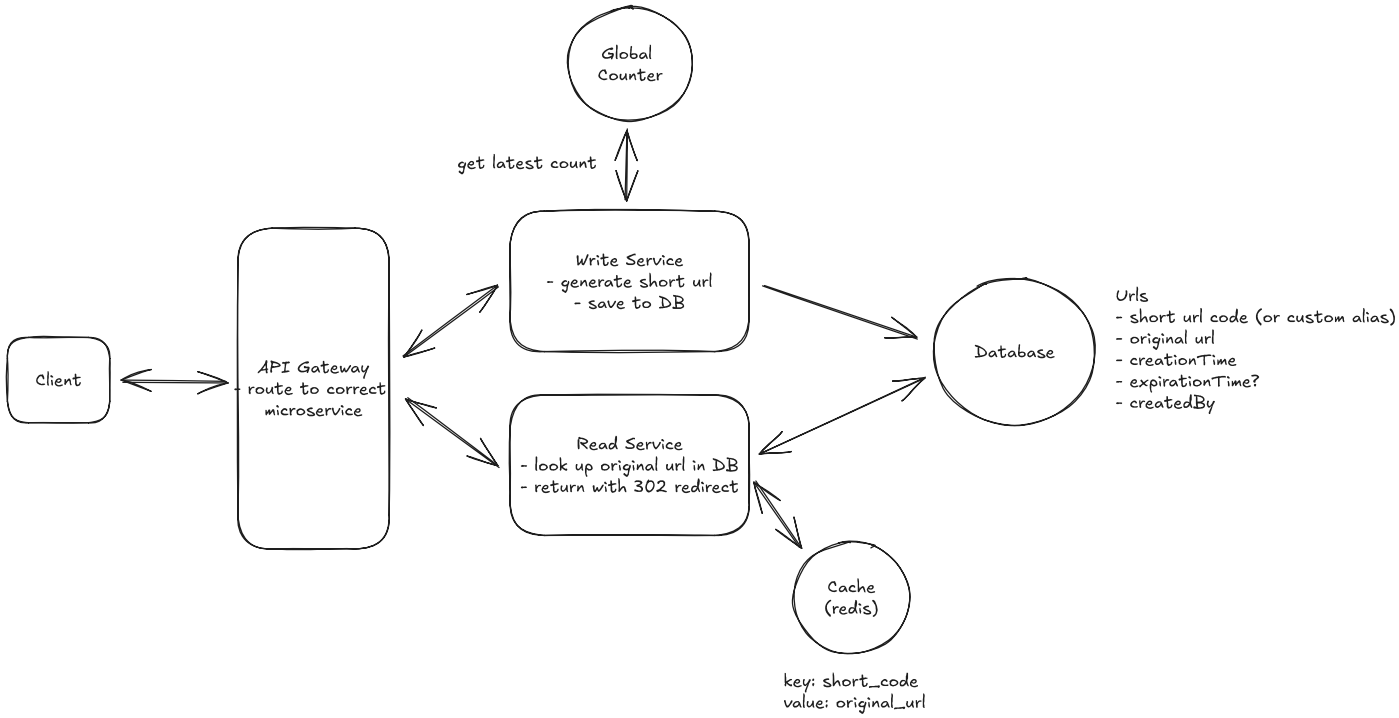
Now, we can horizontally scale both the Read Service and the Write Service to handle increased load. Horizontal scaling is the process of adding more instances of a service to distribute the load across multiple servers. This can help us handle a large number of requests per second without increasing the load on a single server. When a new request comes in, it is randomly routed to one of the instances of the service.

But what about our counter?

Horizontally scaling our write service introduces a significant issue! For our short code generation to remain globally unique, we need a single source of truth for the counter. This

counter needs to be accessible to all instances of the Write Service so that they can all agree on the next value.

We could solve this by using a centralized Redis instance to store the counter. This Redis instance can be used to store the counter and any other metadata that needs to be shared across all instances of the Write Service. Redis is single-threaded and is very fast for this use case. It also supports atomic increment operations which allows us to increment the counter without any issues. Now, when a user requests to shorten a url, the Write Service will get the next counter value from the Redis instance, compute the short code, and store the mapping in the database.



Final Design

But should we be concerned about the overhead of an additional network request for each new write request?

The reality is, this is probably not a big deal. Network requests are fast! In practice, the overhead of an additional network request is negligible compared to the time it takes to perform other operations in the system. That said, we could always use a technique called "counter batching" to reduce the number of network requests. Here's how it works:

1. Each Write Service instance requests a batch of counter values from the Redis instance (e.g., 1000 values at a time).

2. The Redis instance atomically increments the counter by 1000 and returns the start of the batch.
3. The Write Service instance can then use these 1000 values locally without needing to contact Redis for each new URL.
4. When the batch is exhausted, the Write Service requests a new batch.

This approach reduces the load on Redis while still maintaining uniqueness across all instances. It also improves performance by reducing network calls for counter values.

To ensure high availability of our counter service, we can use Redis Sentinel or Redis Cluster with automatic failover. A single Redis instance can handle 100k+ operations per second, far exceeding typical URL shortening rates, especially with counter batching.

For multi-region deployment, allocate disjoint counter ranges to each region (e.g., region A gets 0-1B, region B gets 1B-2B) to avoid cross-region coordination. Writes go to the local region's Redis, while reads can be served globally via distributed caches.

If Redis fails before replicating the latest counter, you might lose a few values, but since we only need uniqueness (not continuity), this is acceptable. The database's UNIQUE constraint on short_code provides the ultimate safety net.

What is Expected at Each Level?

URL shortener is considered an entry-level system design question, but that doesn't mean it's trivial. Here's what I look for at each level.

Mid-level

At mid-level, I expect you to produce a working high-level design that handles URL shortening and redirection. You should understand the basic flow: user submits a long URL, system generates a short code, stores the mapping, and redirects users who visit the short URL. I want to see you recognize that short code generation needs to guarantee uniqueness and propose at least one reasonable approach (hashing or counter-based). You should understand why we use a 302 redirect (even if you did not know the status code yourself) and be able to discuss basic database indexing. With some prompting, you should recognize that a cache would help with read performance given the read-heavy nature of the system.

Senior

For senior candidates, I expect you to drive the conversation and proactively identify the key challenges: unique code generation at scale, fast redirects, and horizontal scaling. You should be able to articulate the tradeoffs between hashing (collision handling) and counter-based approaches (coordination overhead) without much prompting. I expect you to discuss caching strategies in detail, including cache invalidation for expired URLs. You should propose a reasonable database choice and justify it. When we get to scaling, you should recognize that separating read and write services makes sense given the asymmetric workload, and understand how to scale the counter across multiple write instances using Redis or similar coordination.

Staff+

For staff candidates, I'm evaluating your ability to see past the "textbook" solution and discuss real production concerns. You should quickly recognize this is a read-heavy system and structure your design accordingly from the start. I expect you to proactively discuss multi-region deployment, counter range allocation, and what happens during Redis failover without being prompted (if you took the counter batching approach). You should understand the security implications of predictable short codes and propose mitigations if relevant. Staff candidates also demonstrate product thinking by discussing custom alias collision prevention, URL expiration cleanup strategies, and how the system would evolve as requirements change. Rather than just solving the problem, you show you've thought about operating and maintaining the system at scale.

Test Your Knowledge

Answer the question below to find your gaps.

Question 1 of 15

Which is NOT a benefit of caching?

1 Improved response time

2 Reduced database load

3 Lower network latency for repeated requests

4 Increased data consistency